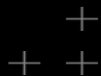


Lists & Strings:

- Fatiamento
- métodos



FATIAMENTO DE LISTAS E STRINGS (SLICING)



Exemplos de Manipulação de Listas

Manipulação e Fatiamento de listas | strings | tuplas

Fatiar lista é selecionar parte de uma lista.

Sintaxe:

```
lista[inicio : fim : passo]
```

fim não o inclui

Onde **inicio** é o primeiro elemento e **fim** o último.

Exemplos:

```
#           0  1  2  3  4  5  6  7  8  9      <= índices
#          -10 -9 -8 -7 -6 -5 -4 -3 -2 -1    <= índices negativos
numeros = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100] # <= valores
print("""
      0  1  2  3  4  5  6  7  8  9      <= índices
      -10 -9 -8 -7 -6 -5 -4 -3 -2 -1    <= índices negativos""")
print(f"Lista = {numeros}")
print(f"3:7   = {numeros[3: 7]}")
print(f"-4:-1 = {numeros[-4:-1]}")
```

✓ 0.0s

```

      0  1  2  3  4  5  6  7  8  9      <= índices
      -10 -9 -8 -7 -6 -5 -4 -3 -2 -1    <= índices negativos
Lista = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
3:7   = [40, 50, 60, 70]
-4:-1 = [70, 80, 90]
```

Modificando os Passos

Manipulando os Passos

Normalmente os passos se dão de 1 em 1, mas podemos modificar para outros valores, veja:

```
#           0  1  2  3  4  5  6  7  8  9      <= índices
#          -10 -9 -8 -7 -6 -5 -4 -3 -2 -1     <= índices negativos
numeros = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100] # <= valores
print("""           0  1  2  3  4  5  6  7  8  9      <= índices
          -10 -9 -8 -7 -6 -5 -4 -3 -2 -1     <= índices negativos""")
print(f"Lista = {numeros}")
print(f"1:8:3 = {numeros[1: 8: 3]}")
print(f"8:1:-2 = {numeros[8: 1: -2]}")
print(f"-3:    = {numeros[-3:]}")
print(f"::-1   = {numeros[::-1]}")
```

✓ 0.0s

```
           0  1  2  3  4  5  6  7  8  9      <= índices
          -10 -9 -8 -7 -6 -5 -4 -3 -2 -1     <= índices negativos
Lista = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
1:8:3 = [20, 50, 80]
8:1:-2 = [90, 70, 50, 30]
-3:    = [80, 90, 100]
::-1   = [100, 90, 80, 70, 60, 50, 40, 30, 20, 10]
```


RECADO IMPORTANTE

O fatiamento de TUPLAS e LISTAS funcionam da mesma forma.

Lembre-se apenas que na declaração da Tupla, os elementos ficam entre parênteses.

Como lidamos com as Tuplas?

Pouco muda, apenas parênteses em tupla no lugar dos colchetes na definição da lista.

```
#           0  1  2  3  4
lista = [1, 2, 3, 4, 5]
tupla = (6, 7, 8, 9, 0)

print(f"Lista: {lista[1:4]}") # Exibe do segundo ao quarto elemento
print(f"Tupla: {tupla[1:4]}") # Exibe do segundo ao quarto elemento
```

✓ 0.0s

```
Lista: [2, 3, 4]
Tupla: (7, 8, 9)
```

Fatiando Strings

String nada mais é do que uma **Lista de caracteres**.

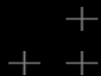
Com strings o mecanismo é o mesmo das listas que vimos até então.

Cada caractere da string corresponde a um elemento da lista string.

```
frase = "Agora veremos como o fatiamento funciona com strings"
print("1. " + frase)
print("2. " + frase[2])
print("3. " + frase[7:15])
print("4. " + frase[::-1])
print("5. " + frase[32:5:-3])
print("6. " + frase[2:20:2])
```

✓ 0.0s

```
1. Agora veremos como o fatiamento funciona com strings
2. o
3. eremos c
4. sgnirts moc anoicnuf otnemaitaf o omoc somerev aroga
5. ftmt ocor
6. oavrmscm
```



MÉTODOS PARA TRATAMIENTO DE STRINGS



Métodos para strings

find(substring, inicio, fim)

Este método procura um trecho de string em uma outra maior, caso encontre, retorna o índice de início da primeira localização, se não encontrar, retorna -1.

```
frase = "Veremos agora os métodos de manipulação de strings."  
print(frase.find("agora")) # Pesquisa a palavra 'agora' na frase  
print(frase.find("m")) # pesquisa a primeira letra 'm' na frase  
print(frase.find("listas")) # pesquisa uma palavra que não existe na frase  
print(frase.find("m",10,20)) # pesquisa a letra 'm' no intervalo entre 10 e 20
```

✓ 0.0s

8

4

-1

17

Métodos para strings

join(lista_de_strings)

O método join() pega uma lista contendo elementos strings e transforma em apenas uma string. Em outras palavras, este método junta strings.

```
nome = ["Edson", "de", "Oliveira"]  
print(" ".join(nome)) # Cria uma string com a lista separando por espaço  
print("-".join(nome)) # Cria uma string com a lista separando com '-'  
print(",".join(nome)) # Cria uma string com a lista separando com ','
```

✓ 0.0s

Edson de Oliveira

Edson-de-Oliveira

Edson,de,Oliveira

Métodos para strings

split(str_separadora)

O método join() pega uma lista contendo elementos strings e transforma em apenas uma string. Em outras palavras, este método junta strings. Repare que ela extrai o texto passado por parâmetro.

```
nome = "Edson de Oliveira"
print(nome.split()) # Tornou a string em uma lista separada por espaço
print(nome.split('e')) # Tornou a string em uma lista separada por 'e'
print(nome.split('de')) # Tornou a string em uma lista separada por 'de'
```

✓ 0.0s

```
['Edson', 'de', 'Oliveira']
['Edson d', ' Oliv', 'ira']
['Edson ', ' Oliveira']
```

Métodos para strings

replace(procura, troca, cont)

Este método substitui em uma string uma substring especificada por outra 'cont' vezes solicitada.

```
nome = "Edson de Oliveira"
nome_new1 = nome.replace('e', 'E') # Troca os 'e' por 'E'
print(nome_new1)
nome_new2 = nome.replace('de', 'DE') # Troca o 'de' por 'DE'
print(nome_new2)
nome_new3 = nome.replace(' ', '_', 1)
print(nome_new3)
```

✓ 0.0s

```
Edson dE OlivEira
Edson DE Oliveira
Edson_de Oliveira
```


Métodos para strings

strip()

Este método elimina os eventuais espaços que existirem no início e final da string.

```
texto = "  Strip elimina os espaços  "
tamanho_texto = len(texto)
print(f"0 texto '{texto}' tem o tamanho: {tamanho_texto}.")
texto = texto.strip() # Elimina os espaços antes e depois do texto
tamanho_texto_strip = len(texto)
print(f"0 texto '{texto}' tem o tamanho: {tamanho_texto_strip}.")
```

✓ 0.0s

0 texto ' Strip elimina os espaços ' tem o tamanho: 30.

0 texto 'Strip elimina os espaços' tem o tamanho: 24.

Operador de identidade 'in'

O operador in retorna True caso o valor comparado esteja contido em uma lista ou False caso não esteja.

```
num = 3
if num in [1, 2, 3, 4, 5]:
    print(f'"{num}" está na lista.')
else:
    print(f'"{num}" NÃO está na lista.')
```

✓ 0.0s

'3' está na lista.

outro exemplo utilizando string:

```
nome = "Edson de Oliveira"
if "de" not in nome:
    print(f'"de" NÃO está contido no nome '{nome}'')
else:
    print(f'"de" ESTÁ contido no nome '{nome}'')
```

✓ 0.0s

'de' ESTÁ contido no nome 'Edson de Oliveira'.

Exercício:

1. Utilizando SOMENTE os métodos apresentados nesta apresentação, faça um subalgoritmo de nome `vogal_maiuscula(string)` que passe uma string por parâmetro e retorne-a com as vogais convertidas em maiúsculas.

Testes:

```
texto = "Edson de Oliveira"  
new_texto = vogal_maiuscula(texto)  
print(new_texto)
```

```
>> EdsOn dE OLIvEIrA
```

Exercício:

2. Dada uma lista e um caractere por parâmetro, faça uma função que retorne em uma lista com os índices onde os caracteres foram encontrados.

Testes:

```
...
#      01234567890123456
Nome = "Edson de Oliveira"
print(retorna_indices(nome, 'e'))
>> [7, 13]
print(retorna_indices(nome, 'a'))
>> [16]
print(retorna_indices(nome, 'm'))
>> []
```


Exercícios:

3. Utilizando SOMENTE os métodos apresentados nesta apresentação, faça uma função chamada `isfloat(string)` que pegue uma string passada por parâmetro e retorne **True** caso esta string represente um valor float ou **False** caso não seja.

Testes:

```
resp = isfloat("Edson")
>> False
resp = isfloat("45.78")
>> True
resp = isfloat("78")
>> False
resp = isfloat("234.")
>> True
resp = isfloat("234,67")
>> False
resp = isfloat("12.567.34")
>> False
resp = isfloat("-12.5")
>> True
```

DESAFIO:

4. Sabemos que o método `isdigit()` e/ou `isnumeric()` analisam uma string passada por parâmetro e retorna `True` caso esta string tenha somente números ou `false` caso um dos caracteres não seja Numérico . Por exemplo

```
x = "123"
print(x.isdigit())
>> True
x = "1A3!"
print(x.isdigit())
>> False
```

Porém a falha deste método é a de retornar `false` caso algum texto com numero inicie com `+` ou `-`.

```
x = "-453"
print(x.isdigit())
>> False
```

Mas, -453 é número!

Aprimore a função `isdigit()` criando uma função chamada: `eh_numero(str) -> bool`: que retorne `True` caso algum texto numérico comece com `+` ou `-`.

```
x = "-453"
print(eh_inteiro(x))
>> True
```

Não use `isdigit` ou `isnumeric` dentro da criação da função